

Chapter 3

A. Questions

1. Please explain the differences between independent and cooperative itineraries.

Answer:

Independent iterations mean that the processes do not affect each other, meaning there is no data shared between them. Conversely, a collaborative itinerary refers to the mutual influence between itineraries, meaning data is shared between them.

2. Why does a collaborative itinerary need a synchronization mechanism?

Answer:

If collaborative processes access shared data simultaneously, it may cause data inconsistency. Therefore, synchronization mechanisms are necessary to ensure that collaborative processes sharing common data can run orderly and maintain data consistency.

3. Explain what is a competition situation. How can we avoid competition situations?

Answer:

When multiple cooperative processes access the same data simultaneously, if different instructions in execution order lead to different results, this situation is called a race situation. To avoid race situations, processes must be synchronized so that only one process can access the shared variable at any given time.

4. Explain what are the critical zone, entry zone, and exit zone.

Answer:

- **Critical Section:** Some sections of the code executed by the cooperative process involve the use of shared resources; These sections cannot allow other processes to access the same shared resource simultaneously during execution, which is called a critical section.
- **Entry Zone:** Before the critical zone, actions are usually taken to obtain permission to enter the critical zone, ensuring that no other process will use the shared resources required by the critical zone during the process's execution.

- **Exit Zone:** The process releases the permissions it originally obtained so that other processes can enter the critical zone and use the shared resources they need.

5. Explain and explain the three conditions that satisfy the critical zone.

Answer:

Mutuality: If a process is in a critical section and the segment accesses a set of shared resources $R_1, R_2, \dots, R_n$, then no other processes can access $R_1, R_2, \dots$, or R_n , in the critical zone of any of these shared resources.

Proceeding: If no iteration is currently in the critical zone and there are iterators waiting to enter the critical zone, one of the waiting iteraries must be able to enter the critical zone within a limited time. If one itinerary ends outside the critical zone, it shouldn't affect other itineraries entering the critical zone.

Limited wait: The time between the request to enter the critical zone and the time granted entry must be limited. Trips cannot be refused entry into the critical zone due to hunger or dead ends.

6. In the philosopher's dinner question, besides the methods listed in this section, explain what other methods can prevent deadlocks.

Answer:

For example, the rule that the last philosopher must first pick up the chopsticks on the left and then the chopsticks on the right; The other philosophers must first pick up the chopsticks on the right and then the left.

7. What is a deadlock? Please explain the four conditions for a deadlock to be established.

Answer:

As long as both sides have certain resources (for example, each crossing half a bridge), they also need other resources in the other's hands (bridges blocked by the other side) to complete the required task (crossing the bridge), which can lead to indefinite waiting and deadlock issues.

8. Please explain how you can break the deadlock? What factors need to be

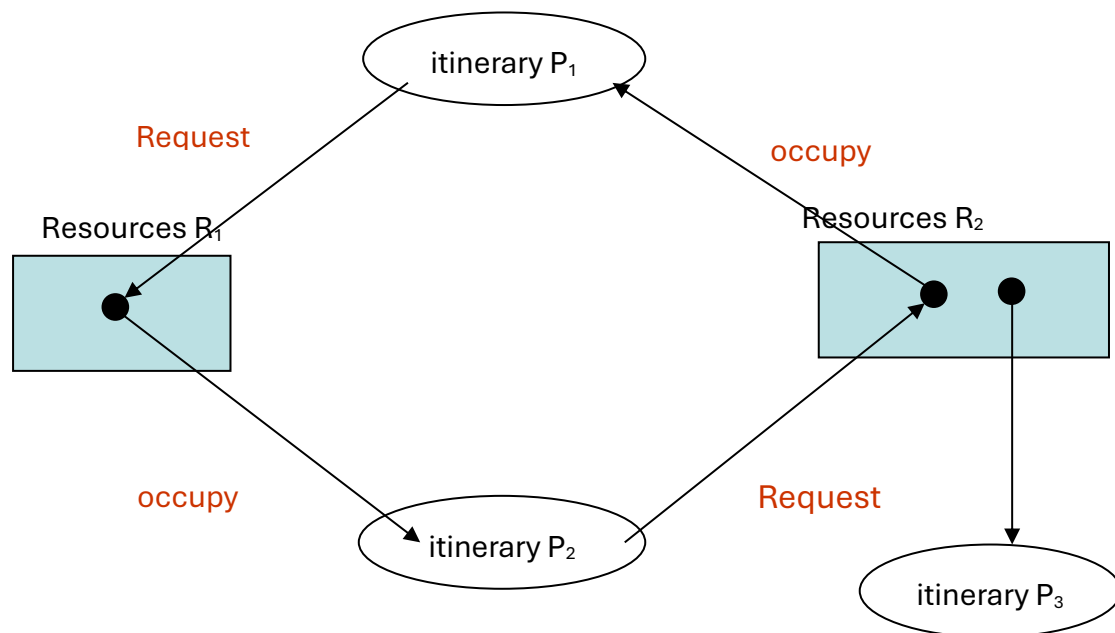
considered?

Answer:

There are two ways to break the deadlock: the first is to directly terminate some processes, making the conditions for loop waiting void; Another method is to recover some resources from the deadlock process to other processes, so that the no-first-priority condition does not apply. Factors to consider include how to choose your itinerary, retrospectives, and hunger phenomena.

9. Suppose there is a closed loop in the resource allocation diagram. Is it inevitable that a dead end will occur?

For example, in the diagram below :



Answer:

Not necessarily. Assuming there are other resources waiting for resources in the closed loop that can satisfy the process, a deadlock may not occur. Although there is a closed loop, because resource R₂ has two resources and process P₁ only occupies one in the loop, after P₃ releases another R₂, P₂ can obtain the resources needed and continue running without a deadlock.

10. Suppose a system has a total of 8 R. Please determine whether the following systems are in a safe state. If so, write a secure execution sequence for it.

itinerary	The total number of R is required	The number of R held
P1	8	1
P2	5	2
P3	1	1
P4	7	2
P5	3	0

Answer:

There is a secure execution sequence (P3, P5, P2, P4, P1), so the system is currently in a secure state.